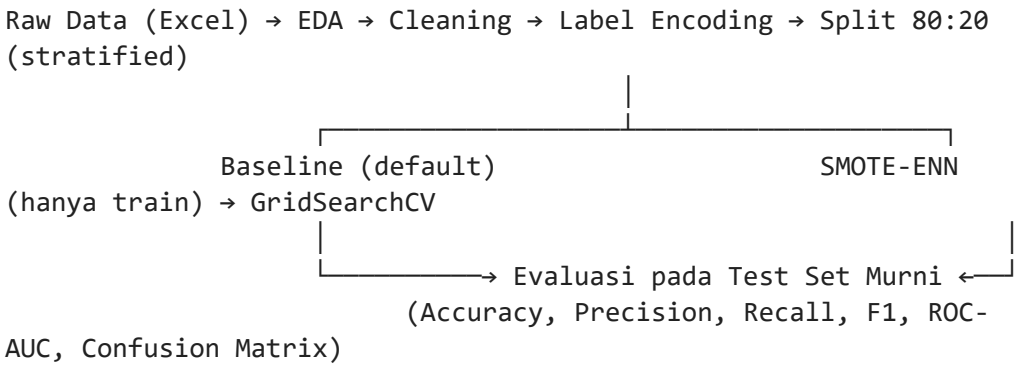
Master Notebook — Prediksi Stunting Balita dengan XGBoost & LightGBM

Notebook ini merupakan **penggabungan end-to-end** seluruh tahap eksperimen skripsi dalam satu alur terpadu: *loading* data mentah, EDA, *cleaning*, *preprocessing*, pemodelan *baseline*, penanganan ketidakseimbangan kelas (SMOTE-ENN), *hyperparameter tuning*, hingga evaluasi dan perbandingan akhir model.

Alur pipeline:



Prinsip metodologi utama:

1. **Anti-data-leakage** — test set (`X_test` / `y_test`) tidak pernah disentuh proses resampling/tuning; hanya dipakai untuk evaluasi akhir. Integritas dijaga *snapshot* + `assert` .
2. **Kolom turunan dibuang** (`Weight for Age` , Z-Score, dll.) karena menurunkan label target.
3. **Kelas positif =** `Stunted` **(1)** — Recall & F1 kelas ini menjadi tolok ukur utama (*false negative* = balita stunting terlewat intervensi).
4. **Reproducibility** — seluruh proses stokastik memakai `random_state=42` .

> **Catatan efisiensi:** seluruh data dialirkan langsung melalui variabel DataFrame di memori (`df` → `df_clean` → `X` / `y` → `X_train` / `X_test`), tanpa file perantara antar tahap.

1. Import Library & Loading Raw Data

Sel pertama berisi **seluruh import library** yang dipakai sepanjang notebook (dikelompokkan per keperluan, tanpa duplikat), diikuti konstanta global dan *loading* data mentah.

- **Sumber data:** `data/raw/Overall Data.xlsx` (gabungan data survei 2021–2024).
- **Target:** kolom `Height for Age` (*Stunted* / *Not Stunted*).
- **Konvensi:** `RANDOM_STATE = 42` untuk reproducibility; kelas positif = `Stunted` (`POS_LABEL = 1`).

In [1]:
```
# ============================== IMPORT LIBRARY ==============================
# --- Manipulasi data & utilitas ---
import pandas as pd
import numpy as np
from datetime import datetime   # deteksi nilai Weight yang terbaca sebagai tanggal
import time                     # pengukuran waktu training model (Bagian 5 & 8)

# --- Visualisasi ---
import matplotlib.pyplot as plt
import seaborn as sns

# --- Splitting, tuning & metrik evaluasi (scikit-learn) ---
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    classification_report,
)

# --- Penanganan class imbalance (imbalanced-learn) ---
from imblearn.combine import SMOTEENN

# --- Model ---
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# --- Setup global ---
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)        # ukuran default figure (width, hei
pd.set_option("display.max_columns", None)      # tampilkan semua kolom dataframe

RANDOM_STATE = 42                               # seed seluruh proses stokastik
TEST_SIZE = 0.20                                # proporsi test set (split 80:20)
LABELS = ["Not Stunted", "Stunted"]             # urutan label kelas (0, 1)
POS_LABEL = 1                                   # kelas positif = Stunted
DATA_PATH = "../data/raw/Overall Data.xlsx"     # sumber data mentah

print("Semua library berhasil di-import. Konfigurasi global siap.")
```

```
Semua library berhasil di-import. Konfigurasi global siap.
```

In [2]:
```
# Loading data mentah — satu-satunya pembacaan file di sepanjang notebook
df = pd.read_excel(DATA_PATH)
```

```
print(f"Shape data mentah: {df.shape[0]:,} baris x {df.shape[1]} kolom\n")
df.dtypes
```

```
Shape data mentah: 40,071 baris x 11 kolom
```

Out[2]:
```
No.                    int64
Gender                   str
Age (Month)          float64
Weight                object
Height               float64
Weight for Age           str
Z-Score  W/A          object
Height for Age           str
Z-Score H/A           object
Weight for Height        str
Z-Score W/H           object
dtype: object
```

In [3]:
```
# Preview 5 baris pertama
df.head()
```

Out[3]:

	No.	Gender	Age (Month)	Weight	Height	Weight for Age	Score	Z-W/A	Height for Age	Z-Score H/A	Weight for Height	S
0	1	F	54.0	13.2	97.5	Normal		-1.94	Stunted	-2.11	Normal	
1	2	M	44.0	12	92.0	Normal		-1.92	Stunted	-2.22	Normal	
2	3	M	57.0	14	97.0	Normal		-1.9	Stunted	-2.58	Normal	
3	4	M	26.0	11	79.0	Normal		-1.15	Stunted	-3.11	Normal	
4	5	F	59.0	14.6	98.0	Normal		-1.66	Stunted	-2.49	Normal	

> **Temuan awal penting:** kolom `Weight` terbaca bercampur tipe data (`datetime` / `int` / `str`) akibat *auto-conversion* Excel — sebagian nilai desimal (mis. `8.2`) terubah menjadi tanggal (`2025-02-08`). Inspeksi detail dilakukan pada Bagian 2 (EDA) dan perbaikannya pada Bagian 3.

2. Exploratory Data Analysis (EDA)

Analisis eksplorasi awal dataset stunting balita. Bagian ini **hanya eksplorasi** dan tidak melakukan perubahan data apa pun.

Target penelitian: kolom `Height for Age` (*Stunted* / *Not Stunted*). Fitur mentah (variabel independen): `Gender` , `Age (Month)` , `Weight` , `Height` .

Fokus pemeriksaan: kelengkapan data (missing value, duplikat, konsistensi kategori), distribusi target & fitur, indikasi outlier, hubungan fitur dengan target, serta data-quality

kolom `Weight`.

2.1 Missing Value per Kolom

In [4]:
```
# Hitung missing value per kolom (hanya tampilkan kolom yang memiliki missing)
missing = df.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
if len(missing) == 0:
    print("Tidak ada missing value di kolom mana pun.")
else:
    print("Kolom dengan missing value:")
    print(missing)
```

```
Kolom dengan missing value:
Weight for Height    5
Age (Month)          2
dtype: int64
```

2.2 Cek Duplikat Baris

In [5]:
```
# Duplikat baris penuh (semua kolom identik)
n_duplicates = df.duplicated().sum()
print(f"Jumlah baris duplikat: {n_duplicates}")
```

```
Jumlah baris duplikat: 0
```

2.3 Nilai Unik & Konsistensi Kategori

Memeriksa nilai unik pada kolom kategorikal untuk mendeteksi kemungkinan inkonsistensi label (spasi berlebih, kapitalisasi, atau variasi penulisan) yang dapat memengaruhi hasil encoding pada tahap preprocessing.

In [6]:
```
# Nilai unik tiap kolom kategorikal
cat_cols = ["Gender", "Height for Age", "Weight for Age", "Weight for Height"]
for col in cat_cols:
    uniq = df[col].dropna().unique()
    print(f"{col} ({len(uniq)} unik): {uniq}\n")
```

```
Gender (2 unik): <StringArray>
['F', 'M']
Length: 2, dtype: str

Height for Age (2 unik): <StringArray>
['Stunted', 'Not Stunted']
Length: 2, dtype: str

Weight for Age (4 unik): <StringArray>
['Normal', 'Underfed', 'Malnutrition', 'Overnutrition']
Length: 4, dtype: str

Weight for Height (4 unik): <StringArray>
['Normal', 'Thin', 'Obese', 'Very Thin']
Length: 4, dtype: str
```

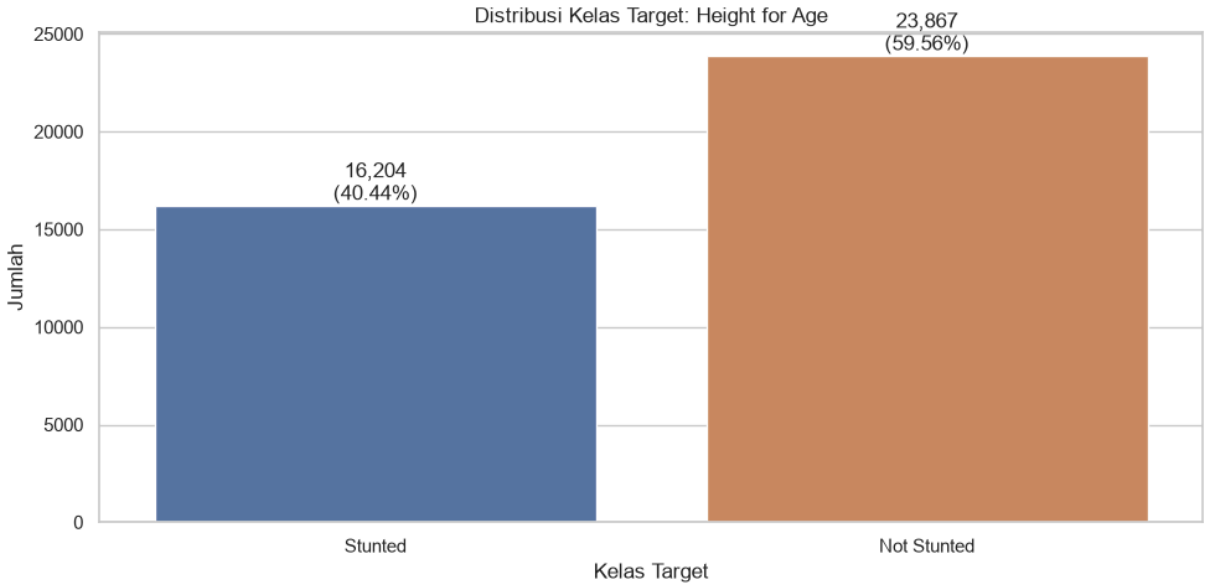
2.4 Distribusi Kelas Target (`Height for Age`)

In [7]:
```python
# Tabel distribusi kelas target (jumlah & persentase)
target_counts = df["Height for Age"].value_counts(dropna=False)
target_pct = df["Height for Age"].value_counts(dropna=False, normalize=True) * 100
target_dist = pd.DataFrame({"count": target_counts, "percentage": target_pct.round(
target_dist
```

Out[7]:

Height for Age	count	percentage
Not Stunted	23867	59.56
Stunted	16204	40.44

In [8]:
```python
# Visualisasi distribusi kelas target dengan anotasi jumlah & persentase
ax = sns.countplot(data=df, x="Height for Age", hue="Height for Age", legend=False)
ax.set_title("Distribusi Kelas Target: Height for Age")
ax.set_xlabel("Kelas Target")
ax.set_ylabel("Jumlah")
for p in ax.patches:
    ax.annotate(f"{int(p.get_height()):,}\n({p.get_height()/len(df)*100:.2f}%)",
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom")
plt.tight_layout()
plt.show()
```



Catatan: Keseimbangan Kelas

Distribusi target relatif **seimbang** (*Not Stunted* ≈ 60% vs *Stunted* ≈ 40%), bukan imbalance ekstrem (rasio ≈ 1.47 : 1). Karena kelas tidak terlalu timpang, manfaat SMOTE-ENN perlu dievaluasi dengan membandingkan metrik **sebelum vs sesudah** resampling (Bagian 5–8).

2.5 Visualisasi Distribusi Fitur Mentah

Fitur: `Gender` (kategorikal), `Age (Month)` & `Height` (numerik). `Weight` **tidak divisualisasikan** karena masih korup (campuran tipe data) — dibahas khusus pada sub-bagian 2.9.

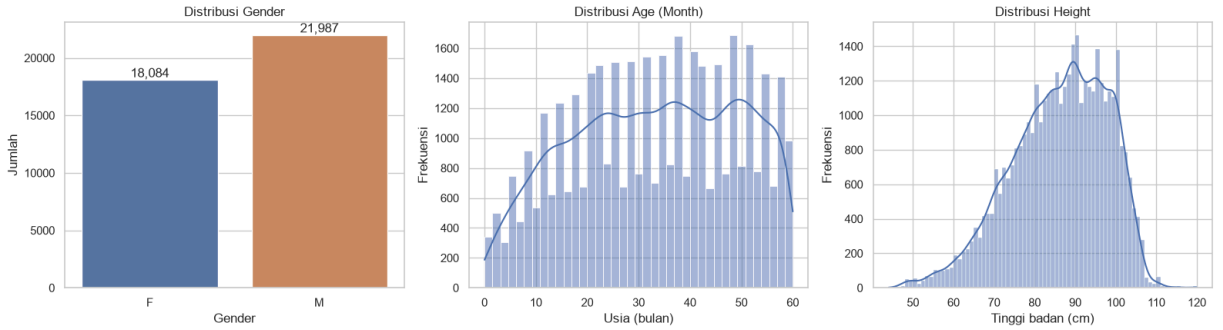
In [9]:
```
# Grid 1x3: distribusi Gender (countplot), Age & Height (histplot + KDE)
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

sns.countplot(data=df, x="Gender", hue="Gender", legend=False, ax=axes[0])
axes[0].set_title("Distribusi Gender")
axes[0].set_xlabel("Gender")
axes[0].set_ylabel("Jumlah")
for p in axes[0].patches:
    axes[0].annotate(f"{int(p.get_height()):,}",
                     (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom")

sns.histplot(df["Age (Month)"].dropna(), kde=True, ax=axes[1])
axes[1].set_title("Distribusi Age (Month)")
axes[1].set_xlabel("Usia (bulan)")
axes[1].set_ylabel("Frekuensi")

sns.histplot(df["Height"].dropna(), kde=True, ax=axes[2])
axes[2].set_title("Distribusi Height")
axes[2].set_xlabel("Tinggi badan (cm)")
axes[2].set_ylabel("Frekuensi")
```

```
plt.tight_layout()
plt.show()
```



2.6 Statistik Deskriptif Fitur Numerik

Ringkasan statistik `Age (Month)` dan `Height`. Kolom `Weight` sengaja dikecualikan karena masih korup pada data mentah; statistiknya baru sahih setelah perbaikan pada Bagian 3.

In [10]:
```
# Statistik deskriptif fitur numerik yang valid
df[["Age (Month)", "Height"]].describe().T
```

Out[10]:

	count	mean	std	min	25%	50%	75%	max
Age (Month)	40069.0	33.020664	15.823079	0.0	20.0	34.0	47.0	60.0
Height	40071.0	86.051172	11.816466	44.0	78.0	87.3	95.2	120.0

2.7 Eksplorasi Outlier (Age & Height)

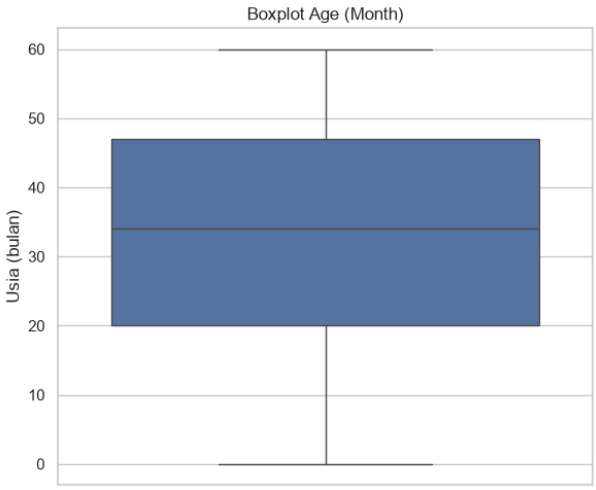
Boxplot untuk mendeteksi nilai ekstrem. EDA di sini **hanya mendeteksi**, bukan menghapus; keputusan penanganan outlier ditunda ke tahap *cleaning* (Bagian 3). `Weight` dikecualikan karena masih korup pada data mentah.

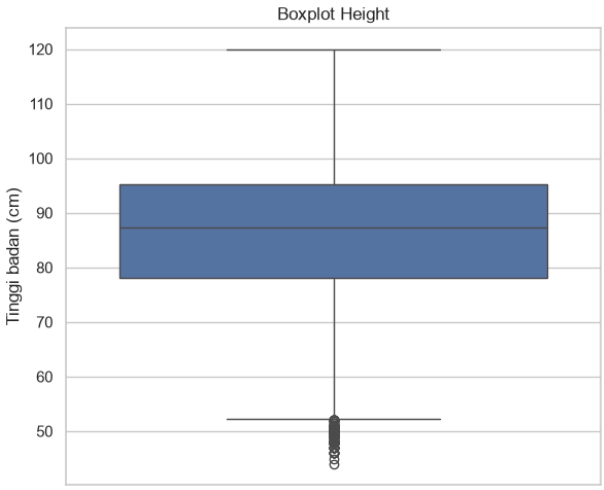
In [11]:
```
# Boxplot Age & Height untuk deteksi outlier awal
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(data=df, y="Age (Month)", ax=axes[0])
axes[0].set_title("Boxplot Age (Month)")
axes[0].set_ylabel("Usia (bulan)")

sns.boxplot(data=df, y="Height", ax=axes[1])
axes[1].set_title("Boxplot Height")
axes[1].set_ylabel("Tinggi badan (cm)")

plt.tight_layout()
plt.show()
```





2.8 Hubungan Fitur dengan Target (`Height for Age`)

Melihat apakah fitur mentah mampu membedakan kelas *Stunted* vs *Not Stunted*. `Weight` dikecualikan (masih korup); fokus pada `Age (Month)`, `Height`, dan `Gender`.

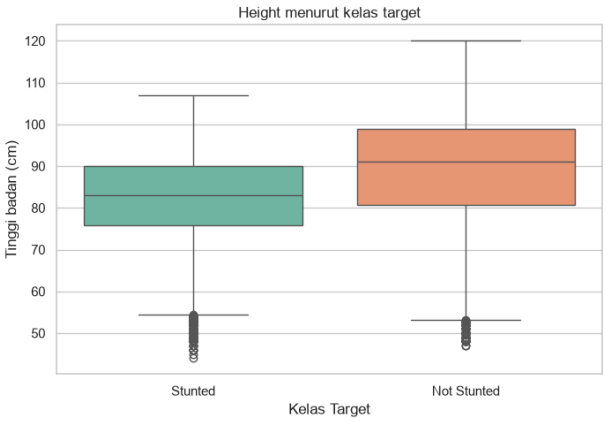
In [12]:
```
# Boxplot Age & Height menurut kelas target (indikasi daya pisah fitur)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(data=df, x="Height for Age", y="Age (Month)", hue="Height for Age",
            legend=False, palette="Set2", ax=axes[0])
axes[0].set_title("Age (Month) menurut kelas target")
axes[0].set_xlabel("Kelas Target")
axes[0].set_ylabel("Usia (bulan)")

sns.boxplot(data=df, x="Height for Age", y="Height", hue="Height for Age",
            legend=False, palette="Set2", ax=axes[1])
axes[1].set_title("Height menurut kelas target")
axes[1].set_xlabel("Kelas Target")
axes[1].set_ylabel("Tinggi badan (cm)")

plt.tight_layout()
plt.show()
```





In [13]:
```
# Statistik Age & Height per kelas target
df.groupby("Height for Age")[["Age (Month)", "Height"]].agg(["count", "mean", "medi
```

Out[13]:

	Age (Month)				Height			
	count	mean	median	std	count	mean	median	std
Height for Age								
Not Stunted	23865	33.651372	35.0	16.501899	23867	88.720644	91.0	12.135873
Stunted	16204	32.091767	32.0	14.717976	16204	82.119285	83.0	10.119848

Gender × Target

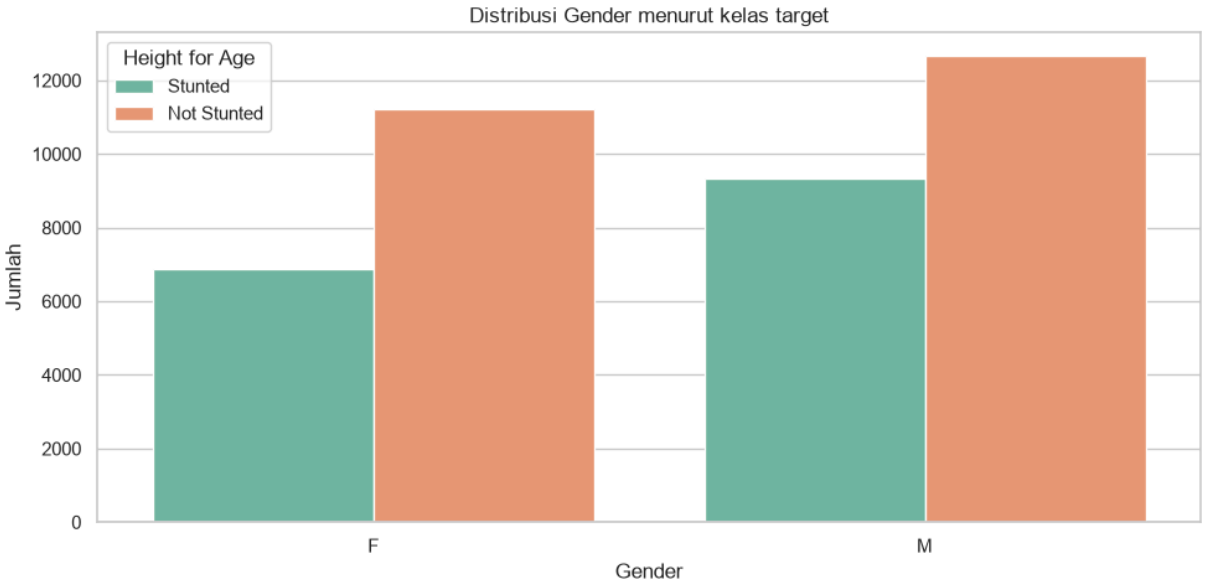
Crosstab jumlah dan persentase (*row-normalized*) untuk melihat apakah proporsi stunting berbeda menurut gender.

In [14]:
```
# Crosstab Gender × Target: jumlah absolut dan persentase per baris
display(pd.crosstab(df["Gender"], df["Height for Age"], margins=True))
display(pd.crosstab(df["Gender"], df["Height for Age"], normalize="index").mul(100)
```

Height for Age	Not Stunted	Stunted	All
Gender			
F	11207	6877	18084
M	12660	9327	21987
All	23867	16204	40071

Height for Age	Not Stunted	Stunted
Gender		
F	61.97	38.03
M	57.58	42.42

In [15]:
```
# Visualisasi proporsi kelas target menurut gender
ax = sns.countplot(data=df, x="Gender", hue="Height for Age", palette="Set2")
ax.set_title("Distribusi Gender menurut kelas target")
ax.set_xlabel("Gender")
ax.set_ylabel("Jumlah")
plt.tight_layout()
plt.show()
```



2.9 Inspeksi Data-Quality Kolom `Weight`

Kolom `Weight` dicurigai rusak akibat *auto-conversion* Excel: sebagian nilai numerik (misalnya `8.2`) terbaca sebagai tanggal `datetime`. Berikut diperiksa **tipe data tiap nilai** di kolom ini (int / str / datetime) tanpa melakukan perbaikan.

In [16]:
```
# Distribusi tipe data per nilai pada kolom Weight
weight_dtype_counts = df["Weight"].map(type).value_counts()
print("Distribusi tipe data per nilai pada kolom Weight:")
print(weight_dtype_counts)

# Rincian jumlah & persentase tiap tipe
is_datetime = df["Weight"].map(lambda x: isinstance(x, datetime))
is_int = df["Weight"].map(lambda x: isinstance(x, int))
is_str = df["Weight"].map(lambda x: isinstance(x, str))
print(f"\nTotal nilai Weight: {len(df):,}")
print(f"  - datetime: {is_datetime.sum():,} ({is_datetime.sum()/len(df)*100:.2f}%)"
print(f"  - int     : {is_int.sum():,} ({is_int.sum()/len(df)*100:.2f}%)")
print(f"  - str     : {is_str.sum():,} ({is_str.sum()/len(df)*100:.2f}%)")
```

```
Distribusi tipe data per nilai pada kolom Weight:
Weight
<class 'datetime.datetime'>    20158
<class 'int'>                  10780
<class 'str'>                   9133
Name: count, dtype: int64

Total nilai Weight: 40,071
  - datetime: 20,158 (50.31%)
  - int     : 10,780 (26.90%)
  - str     : 9,133 (22.79%)
```

In [17]:
```
# Contoh nilai Weight yang terbaca sebagai datetime & sebagai string
print("Contoh nilai Weight yang terbaca sebagai datetime:")
display(df.loc[is_datetime, "Weight"].head(10))
```

```
print("Contoh nilai Weight yang terbaca sebagai string:")
display(df.loc[is_str, "Weight"].head(10))
```

```
Contoh nilai Weight yang terbaca sebagai datetime:
12300    2025-02-08 00:00:00
12301    2025-07-14 00:00:00
12303    2025-05-12 00:00:00
12304    2025-04-10 00:00:00
12306    2025-06-16 00:00:00
12310    2025-02-13 00:00:00
12312    2025-04-15 00:00:00
12313    2025-09-13 00:00:00
12314    2025-05-15 00:00:00
12315    2025-08-14 00:00:00
Name: Weight, dtype: object
Contoh nilai Weight yang terbaca sebagai string:
0     13.2
4     14.6
8     14.2
10    11.5
11    13.1
13    11.1
14    11.1
16     9.5
17    14.7
18    11.2
Name: Weight, dtype: object
```

2.10 Ringkasan Temuan EDA

1. Dataset **40.071 baris × 11 kolom**, tanpa duplikat.
2. Missing value minor: `Age (Month)` (2 nilai) dan `Weight for Height` (5 nilai).
3. Target `Height for Age` relatif seimbang: *Not Stunted* 59.56% vs *Stunted* 40.44%.
4. Kolom `Weight` **korup** (≈ 50% terbaca *datetime* akibat *auto-conversion* Excel).
5. Dari boxplot, `Height` dan `Age` tampak berbeda antara *Stunted*/*Not Stunted* — indikasi awal kedua fitur tersebut informatif.
6. Kolom turunan (`Weight for Age`, `Z-Score`, dll.) berpotensi *data leakage* dan akan dibuang pada tahap preprocessing (Bagian 4).

3. Data Cleaning & Outlier Handling (Perbaikan Kolom Weight dan Penanganan Missing Values)

Bagian ini membersihkan data mentah menjadi data siap-pakai secara **langsung di memori** (`df` → `df_clean`), tanpa menyimpan/memuat file perantara. Tahapan:

1. **Perbaikan kolom `Weight`** — kerusakan akibat *auto-conversion* Excel (datetime → desimal).

2. **Penanganan outlier** `Weight` — koreksi nilai skala gram & penghapusan nilai mustahil.
3. **Penanganan missing value** pada `Age (Month)` dan `Weight for Height`.
4. **Validasi akhir** — memastikan data bersih sepenuhnya sebelum masuk preprocessing.

> Catatan: `df` (data mentah) sengaja dibiarkan utuh sebagai pembanding *before/after*.

3.1 Perbaikan Kolom `Weight`

Kolom `Weight` tercampur 3 tipe data akibat *auto-conversion* Excel:

- `datetime` → nilai desimal asli (misal `2025-02-08` → `8.2`, yaitu format `day.month`, umum di *locale* Indonesia/Eropa),
- `int` / `float` / `str` numerik → dikonversi ke `float`.

Aturan konversi datetime: `float(f"{day}.{month}")`.

In [18]:
```
print("Sebelum perbaikan — tipe data per nilai pada kolom Weight:")
print(df["Weight"].map(type).value_counts())

# Fungsi konversi nilai Weight campuran tipe menjadi float (kg)
def fix_weight_value(x):
    if isinstance(x, datetime):            # tanggal Excel → desimal asli (format d
        return float(f"{x.day}.{x.month}")
    if isinstance(x, bool):
        return float(x)
    if isinstance(x, (int, float)):        # numerik → float
        return float(x)
    if isinstance(x, str):                 # string numerik → float
        return float(x.strip())
    return np.nan                          # tipe tak dikenali → NaN

# Terapkan perbaikan pada salinan data (df mentah tetap utuh sebagai pembanding)
df_clean = df.copy()
df_clean["Weight"] = df_clean["Weight"].apply(fix_weight_value)
print("\nDtype Weight setelah konversi:", df_clean["Weight"].dtype)
print("Jumlah NaN baru pada Weight:", df_clean["Weight"].isna().sum())
```

```
Sebelum perbaikan — tipe data per nilai pada kolom Weight:
Weight
<class 'datetime.datetime'>    20158
<class 'int'>                  10780
<class 'str'>                   9133
Name: count, dtype: int64

Dtype Weight setelah konversi: float64
Jumlah NaN baru pada Weight: 0
```

3.2 Validasi Rentang Weight (1–30 kg)

Setelah konversi, cek apakah semua nilai berada pada rentang wajar berat badan balita (±1–30 kg).

In [19]:
```
# Deteksi nilai Weight di luar rentang wajar balita
low_mask = df_clean["Weight"] < 1
high_mask = df_clean["Weight"] > 30
print(f"Weight < 1 kg : {low_mask.sum()}")
print(f"Weight > 30 kg: {high_mask.sum()}")
print("Nilai Weight > 30 kg:", sorted(df_clean.loc[high_mask, "Weight"].unique()))
```

```
Weight < 1 kg : 0
Weight > 30 kg: 6
Nilai Weight > 30 kg: [np.float64(91.0), np.float64(2800.0), np.float64(2900.0), np.
float64(3000.0), np.float64(3085.0), np.float64(3100.0)]
```

Penanganan Outlier Weight

Validasi menemukan dua jenis nilai di luar rentang wajar:

1. **Nilai skala gram** (`2800, 2900, 3000, 3085, 3100`) — berat bayi baru lahir yang tercatat dalam satuan **gram** (2,8–3,1 kg adalah berat lahir normal). Perbaikan: **bagi 1000**.
2. **Nilai** `91` — mustahil secara biologis untuk balita dan nilai aslinya tidak dapat dipastikan. Karena hanya 1 baris, baris ini **dihapus** (*drop*) daripada ditebak.

In [20]:
```
# Koreksi nilai skala gram → kg (dibagi 1000)
gram_mask = df_clean["Weight"] >= 1000
n_gram = gram_mask.sum()
df_clean.loc[gram_mask, "Weight"] = df_clean.loc[gram_mask, "Weight"] / 1000
print(f"Nilai skala gram yang dikoreksi (dibagi 1000): {n_gram}")
print("Nilai setelah koreksi:", df_clean.loc[gram_mask, "Weight"].tolist())
```

```
Nilai skala gram yang dikoreksi (dibagi 1000): 5
Nilai setelah koreksi: [3.085, 3.1, 2.9, 3.0, 2.8]
```

In [21]:
```
# Drop baris dengan Weight di luar rentang wajar (mustahil secara biologis)
implausible_mask = (df_clean["Weight"] < 1) | (df_clean["Weight"] > 30)
print("Baris dengan Weight di luar 1–30 kg (akan di-drop):", implausible_mask.sum()
display(df_clean.loc[implausible_mask, ["No.", "Gender", "Age (Month)", "Weight", "

df_clean = df_clean.loc[~implausible_mask].reset_index(drop=True)
print("Shape setelah drop outlier Weight:", df_clean.shape)
```

```
Baris dengan Weight di luar 1–30 kg (akan di-drop): 1
```

	No.	Gender	Age (Month)	Weight	Height	Weight for Height
26336	26337	F	28.0	91.0	84.0	NaN

```
Shape setelah drop outlier Weight: (40070, 11)
```

3.3 Penanganan Missing Value

Strategi: drop baris (bukan imputasi).

- `Age (Month)` — 2 nilai hilang (0,005% dari 40.071). Usia adalah fitur numerik kunci; imputasi median/mean akan menyuntikkan nilai sintetis yang tidak berasal dari pengukuran, sedangkan *drop* hanya kehilangan data yang dapat diabaikan → **drop**.
- `Weight for Height` — 5 nilai hilang. Kolom ini adalah **fitur turunan** yang sesuai metodologi justru akan dibuang saat feature selection (anti-leakage, Bagian 4), sehingga imputasi tidak memberi manfaat apa pun bagi pemodelan → **drop**.

In [22]:
```
print("Missing value sebelum drop:")
print(df_clean[["Age (Month)", "Weight for Height"]].isna().sum())

# Drop baris yang memiliki missing pada Age (Month) atau Weight for Height
missing_mask = df_clean["Age (Month)"].isna() | df_clean["Weight for Height"].isna(
n_dropped_missing = missing_mask.sum()
df_clean = df_clean.loc[~missing_mask].reset_index(drop=True)
print(f"\nBaris di-drop karena missing value: {n_dropped_missing}")
print("Shape setelah drop missing value:", df_clean.shape)
```

```
Missing value sebelum drop:
Age (Month)          2
Weight for Height    4
dtype: int64

Baris di-drop karena missing value: 6
Shape setelah drop missing value: (40064, 11)
```

3.4 Validasi Akhir Data Bersih

Memastikan tidak ada missing value tersisa dan seluruh `Weight` berada pada rentang wajar (1–30 kg).

In [23]:
```
# Validasi akhir: data bersih total sebelum masuk preprocessing
print("Missing value total tersisa:", int(df_clean.isna().sum().sum()))
print("Rentang Weight akhir:", df_clean["Weight"].min(), "-", df_clean["Weight"].ma
print("Weight di luar 1–30 kg:", int(((df_clean["Weight"] < 1) | (df_clean["Weight"
print("Dtype Weight akhir:", df_clean["Weight"].dtype)
print("Dtype Age (Month) akhir:", df_clean["Age (Month)"].dtype)
```

```
Missing value total tersisa: 0
Rentang Weight akhir: 2.0 - 28.5 kg
Weight di luar 1–30 kg: 0
Dtype Weight akhir: float64
Dtype Age (Month) akhir: float64
```

3.5 Ringkasan Before / After Cleaning

In [24]:
```
# Perbandingan before/after — df (mentah, utuh) vs df_clean (tanpa baca ulang file)
summary = pd.DataFrame({
    "Aspek": [
        "Jumlah baris",
```

```
        "Jumlah kolom",
        "Missing Age (Month)",
        "Missing Weight for Height",
        "Tipe campuran kolom Weight",
        "Dtype Weight",
        "Rentang Weight (kg)",
    ],
    "Before": [
        len(df),
        df.shape[1],
        int(df["Age (Month)"].isna().sum()),
        int(df["Weight for Height"].isna().sum()),
        "datetime/int/str",
        str(df["Weight"].dtype),
        "tidak valid (datetime, gram, outlier)",
    ],
    "After": [
        len(df_clean),
        df_clean.shape[1],
        int(df_clean["Age (Month)"].isna().sum()),
        int(df_clean["Weight for Height"].isna().sum()),
        "seragam (numerik)",
        str(df_clean["Weight"].dtype),
        f"{df_clean['Weight'].min():.2f} - {df_clean['Weight'].max():.2f}",
    ],
})
summary
```

Out[24]:

	Aspek	Before	After
0	Jumlah baris	40071	40064
1	Jumlah kolom	11	11
2	Missing Age (Month)	2	0
3	Missing Weight for Height	5	0
4	Tipe campuran kolom Weight	datetime/int/str	seragam (numerik)
5	Dtype Weight	object	float64
6	Rentang Weight (kg)	tidak valid (datetime, gram, outlier)	2.00 - 28.50

In [25]:

```
# Distribusi kelas target setelah cleaning (proporsi harus tetap ±60:40)
print("Distribusi kelas target setelah cleaning:")
vc = df_clean["Height for Age"].value_counts()
print(vc)
print((vc / len(df_clean) * 100).round(2))
```

```
Distribusi kelas target setelah cleaning:
Height for Age
Not Stunted    23864
Stunted        16200
Name: count, dtype: int64
Height for Age
Not Stunted    59.56
Stunted        40.44
Name: count, dtype: float64
```

4. Preprocessing & Data Splitting (Label Encoding & Split 80:20 Sebelum Resampling untuk Mencegah Data Leakage)

Tahap persiapan data sebelum pemodelan:

1. **Feature selection** — hanya memakai 4 fitur mentah (`Gender` , `Age (Month)` , `Weight` , `Height`); kolom turunan dibuang untuk mencegah *data leakage*.
2. **Label encoding** — `Gender` (M → 0, F → 1) dan target `Height for Age` (*Not Stunted* → 0, *Stunted* → 1).
3. **Train-test split 80:20 stratified** (`random_state=42`).

> **Urutan ini kritikal:** split dilakukan **sebelum** resampling (SMOTE-ENN di Bagian 6) agar test set benar-benar mewakili distribusi data asli dan tidak pernah tersentuh proses resampling — inilah kunci pencegahan *data leakage*.

4.1 Feature Selection & Data Leakage

Dataset memiliki kolom turunan: `Weight for Age` , `Z-Score W/A` , `Z-Score H/A` , `Weight for Height` , `Z-Score W/H` .

Kolom-kolom ini **wajib dibuang** karena merupakan hasil perhitungan yang **menurunkan label target** (`Height for Age`). Contoh: target adalah hasil klasifikasi dari `Z-Score H/A` (anak dikategorikan *Stunted* bila H/A Z-score < −2). Bila kolom turunan ikut menjadi fitur, model hanya akan "mencontek jawaban" dari rumus yang sudah ada, sehingga performa menjadi semu dan tidak mencerminkan kemampuan prediksi dari data fisik mentah.

Variabel independen yang dipakai hanyalah 4 fitur mentah: `Gender` , `Age (Month)` , `Weight` , `Height` .

In [26]:
```
TARGET_COL = "Height for Age"
FEATURES = ["Gender", "Age (Month)", "Weight", "Height"]
DERIVED_COLS = [
    "Weight for Age",
```

```
    "Z-Score  W/A",   # perhatikan: spasi ganda sesuai nama kolom asli di Excel
    "Z-Score H/A",
    "Weight for Height",
    "Z-Score W/H",
]

# Buang kolom turunan + No. + target, lalu pastikan sisa kolom == FEATURES (integri
cols_to_drop = DERIVED_COLS + ["No.", TARGET_COL]
df_noderived = df_clean.drop(columns=cols_to_drop)
print("Kolom setelah buang turunan + No. + target:", list(df_noderived.columns))
assert list(df_noderived.columns) == FEATURES, "Fitur X tidak sesuai FEATURES yang
```

```
Kolom setelah buang turunan + No. + target: ['Gender', 'Age (Month)', 'Weight', 'Hei
ght']
```

4.2 Label Encoding

- **Gender**: `M` → 0 (Laki-laki), `F` → 1 (Perempuan).
- **Target `Height for Age`**: `Not Stunted` → 0, `Stunted` → 1 (kelas positif).

In [27]:
```
# Susun X (fitur) dan y (target), lalu label-encoding manual (deterministik)
X = df_clean[FEATURES].copy()
y = df_clean[TARGET_COL].map({"Not Stunted": 0, "Stunted": 1})
X["Gender"] = X["Gender"].map({"M": 0, "F": 1})

print("Distribusi Gender setelah encoding:")
print(X["Gender"].value_counts().sort_index())
print("\nDistribusi target setelah encoding:")
print(y.value_counts().sort_index())
```

```
Distribusi Gender setelah encoding:
Gender
0    21985
1    18079
Name: count, dtype: int64

Distribusi target setelah encoding:
Height for Age
0    23864
1    16200
Name: count, dtype: int64
```

4.3 Train-Test Split (80:20, Stratified)

Data dibagi 80% *training* / 20% *testing* secara **stratified** menurut target `y` agar proporsi kelas (*Stunted* : *Not Stunted*) terjaga di kedua bagian, dengan `random_state=42` agar hasil *reproducible*.

In [28]:
```
# Split 80:20 stratified — dilakukan SEBELUM resampling apa pun (anti-data-leakage)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, stratify=y, random_state=RANDOM_STATE
)
```

```
print("X_train shape:", X_train.shape)
print("X_test  shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test  shape:", y_test.shape)

# SNAPSHOT test set — pembanding untuk memastikan X_test/y_test tidak pernah beruba
# oleh proses resampling/tuning di bagian-bagian berikutnya (kunci anti-data-leakag
X_test_snapshot = X_test.copy(deep=True)
y_test_snapshot = y_test.copy(deep=True)
print("\nSnapshot X_test & y_test disimpan untuk validasi anti-leakage.")
```

```
X_train shape: (32051, 4)
X_test  shape: (8013, 4)
y_train shape: (32051,)
y_test  shape: (8013,)

Snapshot X_test & y_test disimpan untuk validasi anti-leakage.
```

4.4 Distribusi Kelas di `y_train` & `y_test`

Memastikan proporsi kelas (*Stunted* ≈ 40%, *Not Stunted* ≈ 60%) tetap terjaga pada *training set* maupun *test set*. Fungsi `show_dist()` akan dipakai ulang pada Bagian 6 (SMOTE-ENN).

In [29]:
```
# Helper cetak distribusi kelas (dipakai ulang di Bagian 6)
def show_dist(name, target):
    s = pd.Series(target)
    counts = s.value_counts().sort_index()
    pct = (s.value_counts(normalize=True).sort_index() * 100).round(2)
    print(f"{name} (total {len(s):,}):")
    for cls in [0, 1]:
        label = "Not Stunted" if cls == 0 else "Stunted"
        print(f"    {label} ({cls}): {counts.get(cls, 0):,}  ({pct.get(cls, 0):.2f}
    print()

show_dist("y (full)", y)
show_dist("y_train", y_train)
show_dist("y_test", y_test)
```

```
y (full) (total 40,064):
    Not Stunted (0): 23,864  (59.56%)
    Stunted (1): 16,200  (40.44%)

y_train (total 32,051):
    Not Stunted (0): 19,091  (59.56%)
    Stunted (1): 12,960  (40.44%)

y_test (total 8,013):
    Not Stunted (0): 4,773  (59.57%)
    Stunted (1): 3,240  (40.43%)
```

PERHATIAN PENTING — Titik Kunci Anti-Data-Leakage

> **Mulai titik ini, `X_test` dan `y_test` TIDAK BOLEH disentuh oleh proses resampling apa pun** (SMOTE, ENN, SMOTE-ENN, *oversampling/undersampling* lain), dan TIDAK BOLEH dipakai untuk *fitting* scaler/imputer, feature selection, maupun hyperparameter tuning.

- SMOTE-ENN **hanya** diterapkan pada `X_train` / `y_train` (Bagian 6).
- `X_test` / `y_test` dibiarkan murni — hanya disentuh saat `.predict()` dan evaluasi — sehingga metrik mencerminkan performa pada data yang belum pernah dilihat model.
- Integritas dijaga dengan *snapshot* (`X_test_snapshot` / `y_test_snapshot`) + `assert` yang diperiksa ulang di Bagian 6, 7, dan 8.

Ringkasan

- Data mentah **40.071 × 11** → bersih **40.064 × 11** (7 baris dihapus: 1 outlier `Weight=91` , 2 missing `Age (Month)` , 5 missing `Weight for Height` — 1 baris tumpang tindih; 5 nilai gram dikoreksi menjadi kg).
- `X` : 4 fitur mentah (`Gender` 0/1, `Age (Month)` , `Weight` , `Height`); `y` : `Stunted` =1, `Not Stunted` =0.
- Split 80:20 stratified (`random_state=42`) → **train 32.051 / test 8.013**.
- Kolom turunan ber-*leakage* dibuang; *snapshot* test set disiapkan untuk validasi anti-leakage.

5. Baseline Modeling (XGBoost & LightGBM)

Notebook-model **baseline** dilatih dengan ketentuan:

- Data: `X_train` / `y_train` hasil split 80:20 stratified (Bagian 4) — **tanpa SMOTE-ENN**, sesuai kondisi asli (*mild imbalance* 40:60).
- **Hyperparameter default** (belum tuning).
- `random_state=42` hanya untuk reproducibility proses stokastik internal, bukan bagian tuning.
- Evaluasi pada `X_test` / `y_test` murni (tidak pernah diresampling).

Metrik: Confusion Matrix, Accuracy, Precision, Recall, F1-Score, ROC-AUC. **Kelas positif = `Stunted` (1)** — recall kelas ini prioritas karena *false negative* berarti balita stunting terlewat intervensi.

5.1 Konfigurasi Model Baseline (Default Hyperparameter)

In [30]:
```
# Dua model baseline dengan hyperparameter default (random_state hanya untuk reprod
baseline_models = {
    "XGBoost": XGBClassifier(
        random_state=RANDOM_STATE, eval_metric="logloss", verbosity=0
    ),
    "LightGBM": LGBMClassifier(
        random_state=RANDOM_STATE, verbosity=-1
    ),
}
```

5.2 Latih, Evaluasi & Ukur Waktu Training

Fungsi `evaluate_on_test()` di bawah adalah **helper evaluasi tunggal** sepanjang notebook — dipakai baseline (Bagian 5) dan model tuned (Bagian 7) agar perhitungan metrik dijamin identik. Prediksi (`y_pred`) dan probabilitas (`y_proba`) **disimpan di memori** untuk kebutuhan kurva ROC & confusion matrix pada Bagian 8 — sehingga tidak perlu melatih ulang model.

In [31]:
```
# Helper evaluasi tunggal: fit (terukur) + prediksi + seluruh metrik + confusion ma
def evaluate_on_test(name, model, X_tr, y_tr):
    t0 = time.perf_counter()                        # ukur waktu training
    model.fit(X_tr, y_tr)
    train_time = time.perf_counter() - t0

    y_pred = model.predict(X_test)                  # test set murni, hanya untuk pre
    y_proba = model.predict_proba(X_test)[:, 1]     # probabilitas kelas positif (Stu

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
    metrics = {
        "model": name,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, pos_label=POS_LABEL),
        "recall": recall_score(y_test, y_pred, pos_label=POS_LABEL),
        "f1_score": f1_score(y_test, y_pred, pos_label=POS_LABEL),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "tn": tn, "fp": fp, "fn": fn, "tp": tp,
        "train_time_s": train_time,
    }
    return metrics, y_pred, y_proba


# Latih & evaluasi kedua model baseline
baseline_rows, baseline_preds, baseline_probas = [], {}, {}
for name, model in baseline_models.items():
    metrics, y_pred, y_proba = evaluate_on_test(name, model, X_train, y_train)
    baseline_rows.append(metrics)
    baseline_preds[name] = y_pred
    baseline_probas[name] = y_proba

    print(f"===== {name} (baseline) =====")
    print(classification_report(y_test, y_pred, target_names=LABELS, digits=4))
```

```
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
    print(pd.DataFrame(cm, index=[f"Actual {l}" for l in LABELS],
                       columns=[f"Pred {l}" for l in LABELS]))
    print(f"Waktu training: {metrics['train_time_s']:.4f}s\n")
```

```
===== XGBoost (baseline) =====
              precision    recall  f1-score   support

 Not Stunted     0.9599    0.9721    0.9660      4773
     Stunted     0.9582    0.9401    0.9491      3240

    accuracy                         0.9592      8013
   macro avg     0.9590    0.9561    0.9575      8013
weighted avg     0.9592    0.9592    0.9591      8013

                   Pred Not Stunted  Pred Stunted
Actual Not Stunted              4640           133
Actual Stunted                   194          3046
Waktu training: 3.8712s

===== LightGBM (baseline) =====
              precision    recall  f1-score   support

 Not Stunted     0.9632    0.9715    0.9674      4773
     Stunted     0.9575    0.9454    0.9514      3240

    accuracy                         0.9609      8013
   macro avg     0.9604    0.9584    0.9594      8013
weighted avg     0.9609    0.9609    0.9609      8013

                   Pred Not Stunted  Pred Stunted
Actual Not Stunted              4637           136
Actual Stunted                   177          3063
Waktu training: 2.7057s
```

5.3 Tabel Hasil Baseline

In [32]:
```
# Kumpulkan hasil baseline ke satu tabel (in-memory, tanpa simpan CSV)
baseline_results = pd.DataFrame(baseline_rows)
baseline_results
```

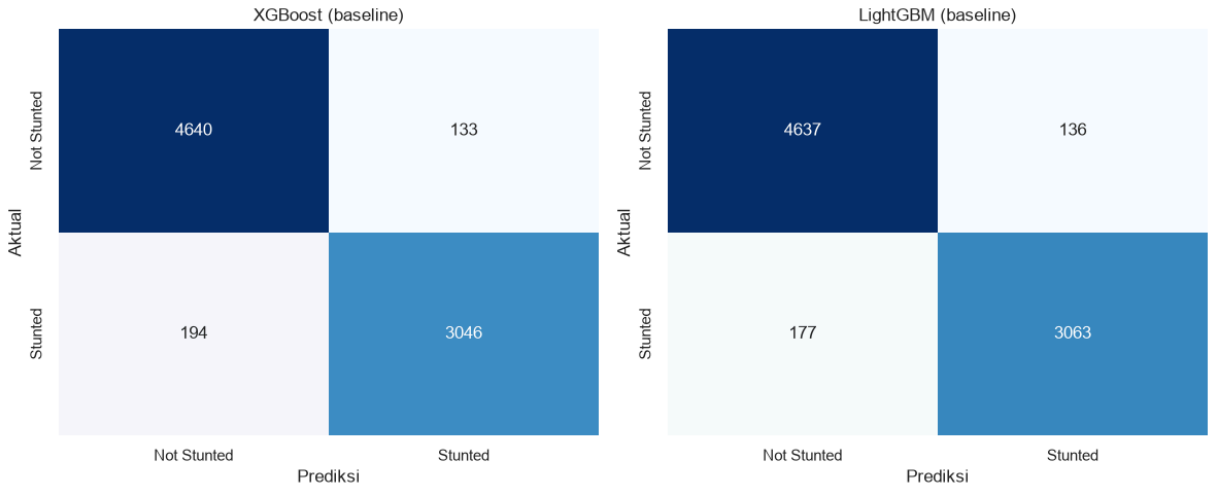
Out[32]:

	model	accuracy	precision	recall	f1_score	roc_auc	tn	fp	fn	tp	trai
0	XGBoost	0.959191	0.958163	0.940123	0.949057	0.992883	4640	133	194	3046	
1	LightGBM	0.960938	0.957487	0.945370	0.951390	0.993038	4637	136	177	3063	

5.4 Confusion Matrix (Visual)

In [33]:
```
# Confusion matrix kedua model baseline
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, name in zip(axes, baseline_models.keys()):
```

```
    cm = confusion_matrix(y_test, baseline_preds[name], labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                xticklabels=LABELS, yticklabels=LABELS, ax=ax)
    ax.set_title(f"{name} (baseline)")
    ax.set_xlabel("Prediksi")
    ax.set_ylabel("Aktual")
plt.tight_layout()
plt.show()
```



6. Handling Class Imbalance (SMOTE-ENN)

Bagian ini menerapkan teknik hybrid resampling **SMOTE-ENN** untuk menangani ketidakseimbangan kelas (*mild imbalance* 40:60) sekaligus membersihkan noise akibat *class overlapping*.

Aturan kritis anti-data-leakage:

- SMOTE-ENN **HANYA** diterapkan pada `X_train` / `y_train`.
- `X_test` / `y_test` **tidak disentuh sama sekali** — divalidasi eksplisit dengan `assert` terhadap *snapshot* (Bagian 4) bahwa ukuran & isinya tidak berubah.

Urutan yang benar: **split (Bagian 4) → SMOTE-ENN pada training set → tuning (Bagian 7) → prediksi pada test set murni (Bagian 8)**.

6.1 Distribusi Kelas Sebelum SMOTE-ENN

In [34]:
```
# Distribusi kelas training set sebelum resampling (helper dari Bagian 4)
show_dist("y_train SEBELUM SMOTE-ENN", y_train)
```

```
y_train SEBELUM SMOTE-ENN (total 32,051):
    Not Stunted (0): 19,091  (59.56%)
    Stunted (1): 12,960  (40.44%)
```

6.2 Terapkan SMOTE-ENN (Hanya pada Training Set)

`SMOTEENN` = **SMOTE** (*oversampling* sintetis kelas minoritas, default `k_neighbors=5`) diikuti **ENN/EditedNearestNeighbours** (pembersih noise, default `n_neighbors=3`). `random_state=42` untuk reproducibility.

In [35]:
```
# Snapshot test set untuk validasi anti-leakage di titik ini
n_test_before = len(X_test)
X_test_pre_smote = X_test.copy(deep=True)
y_test_pre_smote = y_test.copy(deep=True)

# Terapkan SMOTE-ENN HANYA pada training set
smote_enn = SMOTEENN(random_state=RANDOM_STATE)
X_train_resampled, y_train_resampled = smote_enn.fit_resample(X_train, y_train)

# Validasi anti-leakage: test set tidak boleh berubah ukuran maupun isinya
assert len(X_test) == n_test_before, "X_test berubah jumlah barisnya! Jangan resamp
assert X_test.equals(X_test_snapshot), "X_test termodifikasi isinya!"
assert y_test.equals(y_test_snapshot), "y_test termodifikasi!"
assert X_test.equals(X_test_pre_smote), "X_test termodifikasi saat SMOTE-ENN!"
print("Validasi lulus: X_test & y_test tidak berubah (leakage aman).")
```

```
Validasi lulus: X_test & y_test tidak berubah (leakage aman).
```

6.3 Distribusi Kelas Sesudah SMOTE-ENN

In [36]:
```
# Bentuk & distribusi kelas training set setelah SMOTE-ENN
print("Tipe hasil:", type(X_train_resampled).__name__, type(y_train_resampled).__na
print("Shape X_train_resampled:", X_train_resampled.shape)
print("Shape y_train_resampled:", np.asarray(y_train_resampled).shape)
print()
show_dist("y_train SESUDAH SMOTE-ENN", y_train_resampled)

# Verifikasi tambahan: total test set tetap 8.013 baris
assert len(X_test) == 8013, "Ukuran test set berubah!"
print("Test set tetap utuh:", len(X_test), "baris — tidak ikut resampling.")
```

```
Tipe hasil: DataFrame Series
Shape X_train_resampled: (34756, 4)
Shape y_train_resampled: (34756,)

y_train SESUDAH SMOTE-ENN (total 34,756):
    Not Stunted (0): 17,218  (49.54%)
    Stunted (1): 17,538  (50.46%)

Test set tetap utuh: 8013 baris — tidak ikut resampling.
```

7. Hyperparameter Tuning (GridSearchCV dengan StratifiedKFold)

Tuning dilakukan pada **data training hasil SMOTE-ENN** (`X_train_resampled` / `y_train_resampled`) menggunakan **GridSearchCV + StratifiedKFold**.

Kunci anti-data-leakage:

- Hyperparameter tuning hanya di dalam training set (cross-validation di atas data resampled), **tidak pernah** melihat test set.
- Evaluasi final memakai `X_test` / `y_test` asli — sama persis dengan baseline (Bagian 5).

Grid yang di-tuning:

- **XGBoost**: `n_estimators` , `max_depth` , `learning_rate` , `gamma` .
- **LightGBM**: `n_estimators` , `max_depth` , `learning_rate` , `num_leaves` .

Scoring CV = **F1 kelas positif (`Stunted` =1)**, konsisten dengan fokus metodologi (recall/F1 kelas minoritas lebih penting daripada accuracy).

7.1 Setup Grid & Cross-Validation

`StratifiedKFold` (5 fold, shuffle, `random_state=42`) menjaga proporsi kelas di tiap fold selama tuning.

> **Catatan metodologi (batasan):** SMOTE-ENN diterapkan **sekali** pada training set (Bagian 6), lalu GridSearchCV dijalankan di atas data yang sudah resample. Artinya sampel sintetis dapat terpecah antar fold CV, sehingga skor F1 CV cenderung sedikit optimistik. Test set tetap murni (dicek `assert`), jadi metrik final tetap valid; yang berisiko hanya ketepatan pemilihan hyperparameter. Ini dicatat sebagai batasan metodologi (bukan leakage).

In [37]:
```
# Setup cross-validation strategy & grid hyperparameter per model
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

xgb_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "gamma": [0, 0.1, 0.3],
}

lgbm_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "num_leaves": [15, 31, 63],
}
```

7.2 Tuning XGBoost

In [38]:
```
# GridSearchCV XGBoost — 81 kombinasi x 5 fold, scoring F1 kelas Stunted
xgb_base = XGBClassifier(
    random_state=RANDOM_STATE, eval_metric="logloss", verbosity=0
)
xgb_gscv = GridSearchCV(
    xgb_base, xgb_grid, scoring="f1", cv=skf, n_jobs=1, verbose=0
)
xgb_gscv.fit(X_train_resampled, y_train_resampled)

print("Best params XGBoost:", xgb_gscv.best_params_)
print(f"Best CV F1 (mean): {xgb_gscv.best_score_:.4f}")
```

```
Best params XGBoost: {'gamma': 0, 'learning_rate': 0.2, 'max_depth': 7, 'n_estimator
s': 300}
Best CV F1 (mean): 0.9965
```

7.3 Tuning LightGBM

In [39]:
```
# GridSearchCV LightGBM — 81 kombinasi x 5 fold, scoring F1 kelas Stunted
lgbm_base = LGBMClassifier(
    random_state=RANDOM_STATE, verbosity=-1
)
lgbm_gscv = GridSearchCV(
    lgbm_base, lgbm_grid, scoring="f1", cv=skf, n_jobs=1, verbose=0
)
lgbm_gscv.fit(X_train_resampled, y_train_resampled)

print("Best params LightGBM:", lgbm_gscv.best_params_)
print(f"Best CV F1 (mean): {lgbm_gscv.best_score_:.4f}")
```

```
Best params LightGBM: {'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 300, 'n
um_leaves': 63}
Best CV F1 (mean): 0.9969
```

7.4 Evaluasi Model Final pada Test Set

Model terbaik (`best_estimator_`) dievaluasi pada `X_test` / `y_test` **asli** — test set yang sama dengan baseline (Bagian 5). Waktu training model tuned memakai `refit_time_` GridSearchCV (waktu refit model terbaik pada seluruh data resampled), lebih akurat daripada mengukur manual karena mencerminkan training final.

In [40]:
```
# Model final hasil tuning (sudah di-refit otomatis oleh GridSearchCV pada data res
tuned_models = {
    "XGBoost": xgb_gscv.best_estimator_,
    "LightGBM": lgbm_gscv.best_estimator_,
}

# Validasi anti-leakage ulang sebelum evaluasi final
assert X_test.equals(X_test_snapshot), "X_test termodifikasi selama proses tuning!"
```

```python
assert y_test.equals(y_test_snapshot), "y_test termodifikasi selama proses tuning!"
assert len(X_test) == 8013, "Test set tidak sama dengan baseline!"

# Evaluasi model tuned (fit sudah dilakukan GridSearchCV → hanya prediksi & metrik)
tuned_rows, tuned_preds, tuned_probas = [], {}, {}
for name, model in tuned_models.items():
    y_pred = model.predict(X_test)                # test set murni, hanya untuk pre
    y_proba = model.predict_proba(X_test)[:, 1]   # probabilitas kelas positif (Stu

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
    tuned_rows.append({
        "model": name,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, pos_label=POS_LABEL),
        "recall": recall_score(y_test, y_pred, pos_label=POS_LABEL),
        "f1_score": f1_score(y_test, y_pred, pos_label=POS_LABEL),
        "roc_auc": roc_auc_score(y_test, y_proba),
        "tn": tn, "fp": fp, "fn": fn, "tp": tp,
        "train_time_s": xgb_gscv.refit_time_ if name == "XGBoost" else lgbm_gscv.re
    })
    tuned_preds[name] = y_pred
    tuned_probas[name] = y_proba

    print(f"===== {name} (tuned, SMOTE-ENN) =====")
    print(classification_report(y_test, y_pred, target_names=LABELS, digits=4))
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
    print(pd.DataFrame(cm, index=[f"Actual {l}" for l in LABELS],
                       columns=[f"Pred {l}" for l in LABELS]))
    print(f"Waktu refit (GridSearchCV.refit_time_): "
          f"{(xgb_gscv.refit_time_ if name == 'XGBoost' else lgbm_gscv.refit_time_)

tuned_results = pd.DataFrame(tuned_rows)
tuned_results
```

```
===== XGBoost (tuned, SMOTE-ENN) =====
              precision    recall  f1-score   support

 Not Stunted     0.9629    0.9633    0.9631      4773
     Stunted     0.9460    0.9454    0.9457      3240

    accuracy                         0.9561      8013
   macro avg     0.9544    0.9544    0.9544      8013
weighted avg     0.9561    0.9561    0.9561      8013

                    Pred Not Stunted  Pred Stunted
Actual Not Stunted              4598           175
Actual Stunted                   177          3063
Waktu refit (GridSearchCV.refit_time_): 0.2472s

===== LightGBM (tuned, SMOTE-ENN) =====
              precision    recall  f1-score   support

 Not Stunted     0.9654    0.9600    0.9627      4773
     Stunted     0.9415    0.9494    0.9454      3240

    accuracy                         0.9557      8013
   macro avg     0.9535    0.9547    0.9541      8013
weighted avg     0.9558    0.9557    0.9557      8013

                    Pred Not Stunted  Pred Stunted
Actual Not Stunted              4582           191
Actual Stunted                   164          3076
Waktu refit (GridSearchCV.refit_time_): 0.2449s
```

Out[40]:

	model	accuracy	precision	recall	f1_score	roc_auc	tn	fp	fn	tp	trai
0	XGBoost	0.956071	0.945954	0.945370	0.945662	0.990805	4598	175	177	3063	
1	LightGBM	0.955697	0.941537	0.949383	0.945443	0.990885	4582	191	164	3076	

8. Model Evaluation & Comparison

Perbandingan **4 kombinasi** (semua dievaluasi pada test set murni yang sama):

1. XGBoost baseline (default, tanpa resampling)
2. XGBoost + SMOTE-ENN (tuned)
3. LightGBM baseline (default, tanpa resampling)
4. LightGBM + SMOTE-ENN (tuned)

> **PENTING (catatan interpretasi):** jangan memilih model "terbaik" hanya dari **accuracy**. Untuk kasus kesehatan dengan kelas tidak seimbang, fokus pada **Recall** dan **F1-Score kelas** `Stunted` — *false negative* (balita stunting terprediksi normal) berakibat fatal: anak tidak mendapat intervensi gizi dini.

In [41]:
```
# Gabungkan hasil baseline & tuned in-memory (tanpa baca CSV antar notebook)
base = baseline_results.copy()
base["scenario"] = "baseline"
sme = tuned_results.copy()
sme["scenario"] = "SMOTE-ENN"
combined = pd.concat([base, sme], ignore_index=True)
combined["label"] = combined["model"] + " + " + combined["scenario"]

# Validasi integritas: test set tetap murni hingga akhir notebook
assert X_test.equals(X_test_snapshot) and y_test.equals(y_test_snapshot), \
    "Test set termodifikasi — hasil tidak valid!"
assert len(X_test) == 8013
print("Final check: test set tetap identik dengan snapshot (anti-leakage terjaga)."
print("Baris data yang dibandingkan:", len(combined))
```

```
Final check: test set tetap identik dengan snapshot (anti-leakage terjaga).
Baris data yang dibandingkan: 4
```

8.1 Tabel Perbandingan 4 Kombinasi

In [42]:
```
# Tabel perbandingan final: metrik utama + waktu training
table = combined[[
    "label", "accuracy", "precision", "recall", "f1_score", "roc_auc", "train_time_
]].copy()
table.columns = [
    "Kombinasi", "Accuracy", "Precision", "Recall (Stunted)", "F1 (Stunted)",
    "ROC-AUC", "Train time (s)"
]
table = table.round(4)
table
```

Out[42]:

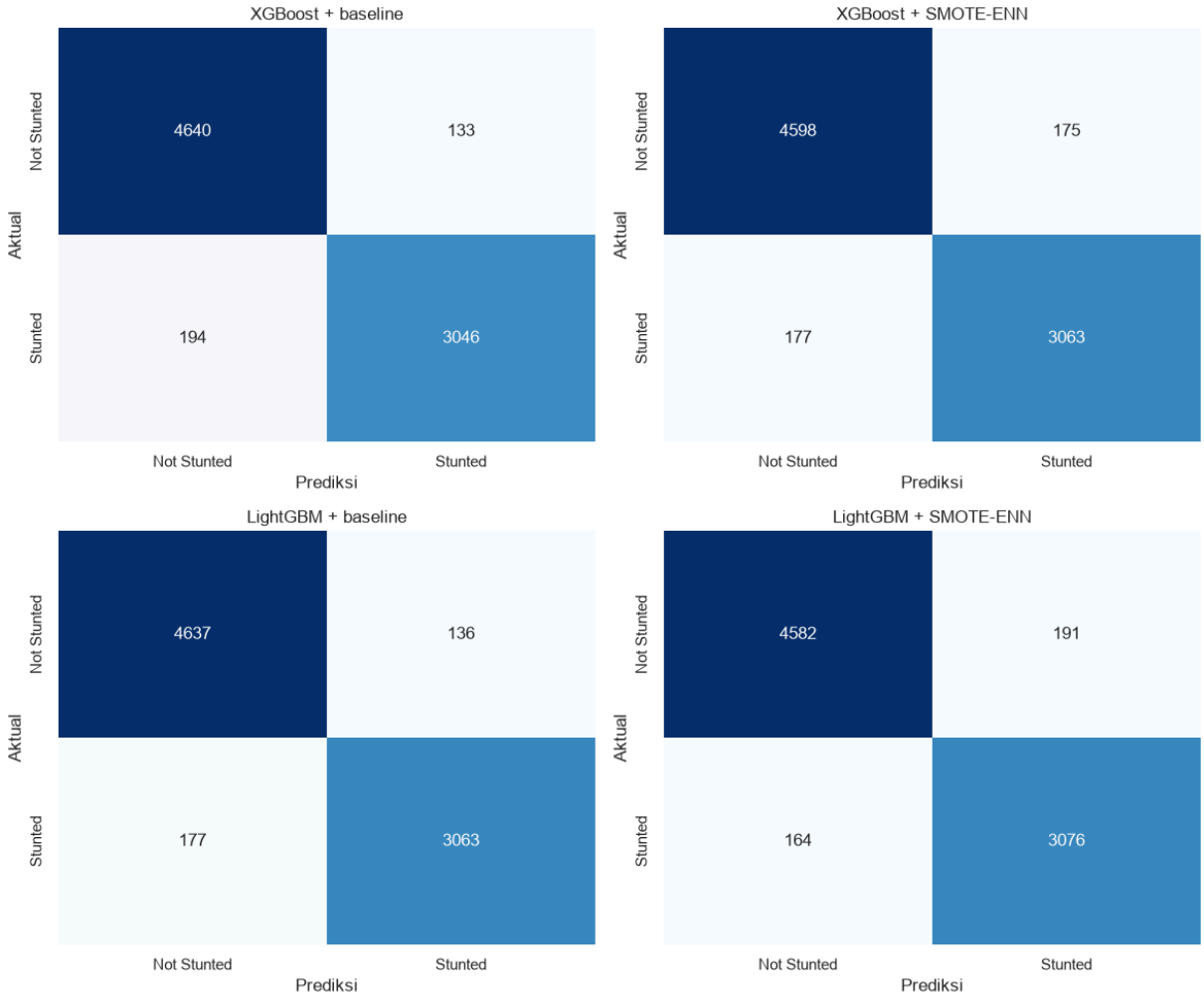
	Kombinasi	Accuracy	Precision	Recall (Stunted)	F1 (Stunted)	ROC-AUC	Train time (s)
0	XGBoost + baseline	0.9592	0.9582	0.9401	0.9491	0.9929	3.8712
1	LightGBM + baseline	0.9609	0.9575	0.9454	0.9514	0.9930	2.7057
2	XGBoost + SMOTE-ENN	0.9561	0.9460	0.9454	0.9457	0.9908	0.2472
3	LightGBM + SMOTE-ENN	0.9557	0.9415	0.9494	0.9454	0.9909	0.2449

8.2 Confusion Matrix (4 Kombinasi)

In [43]:
```
# Confusion matrix 2x2 untuk seluruh kombinasi (prediksi dari memori, tanpa refit)
configs_order = [
    "XGBoost + baseline", "XGBoost + SMOTE-ENN",
    "LightGBM + baseline", "LightGBM + SMOTE-ENN",
]
```

```
preds_all = {
    "XGBoost + baseline": baseline_preds["XGBoost"],
    "XGBoost + SMOTE-ENN": tuned_preds["XGBoost"],
    "LightGBM + baseline": baseline_preds["LightGBM"],
    "LightGBM + SMOTE-ENN": tuned_preds["LightGBM"],
}

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
for ax, label in zip(axes.ravel(), configs_order):
    cm = confusion_matrix(y_test, preds_all[label], labels=[0, 1])
    s = sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                    xticklabels=LABELS, yticklabels=LABELS, ax=ax)
    ax.set_title(label)
    ax.set_xlabel("Prediksi")
    ax.set_ylabel("Aktual")
plt.tight_layout()
plt.show()
```



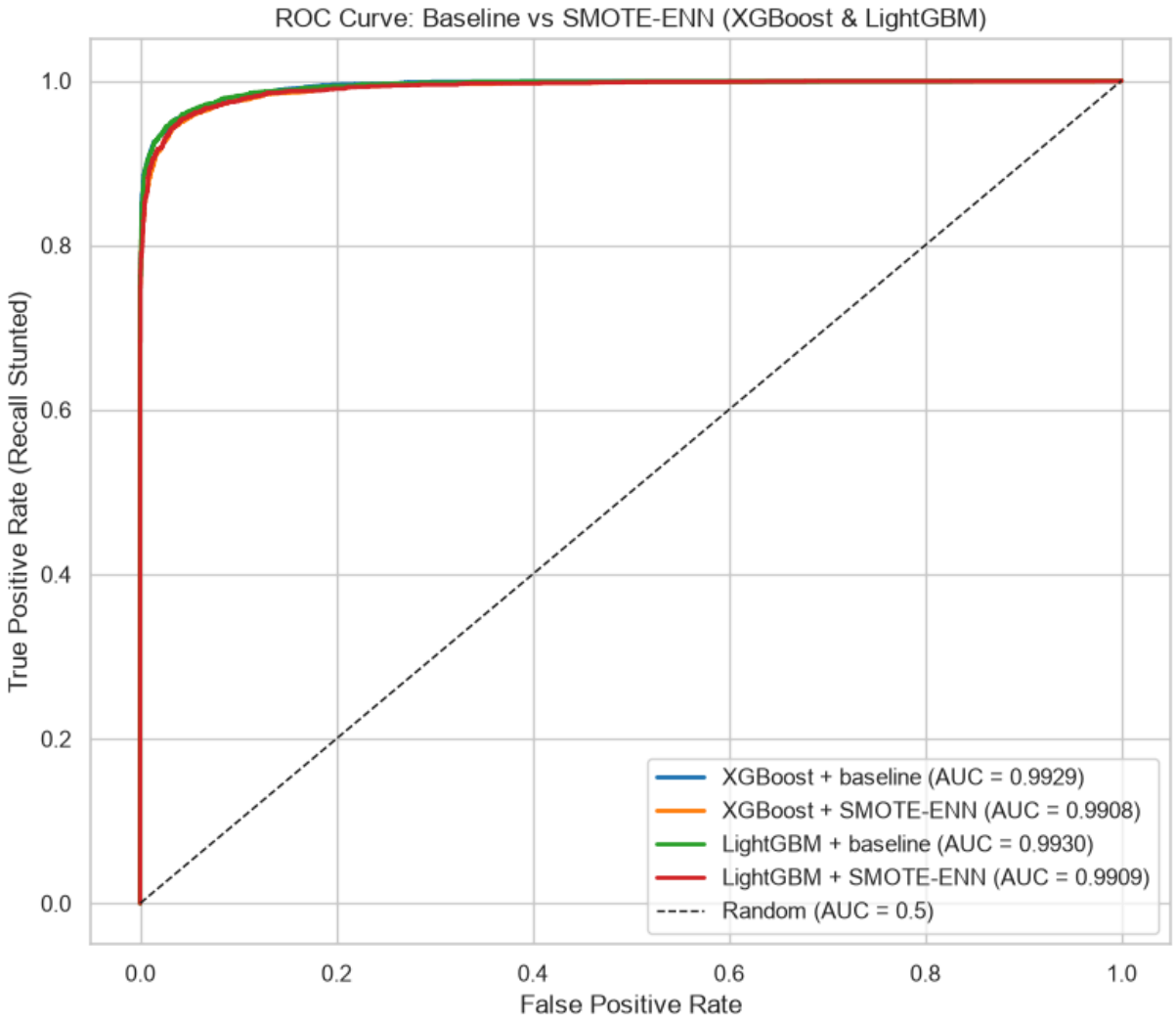
8.3 ROC Curve (Overlay 4 Model)

In [44]:
```
# Kurva ROC keempat model — probabilitas dari memori (tanpa refit model)
color_map = {
    "XGBoost + baseline": "tab:blue",
    "XGBoost + SMOTE-ENN": "tab:orange",
```

```
    "LightGBM + baseline": "tab:green",
    "LightGBM + SMOTE-ENN": "tab:red",
}
probas_all = {
    "XGBoost + baseline": baseline_probas["XGBoost"],
    "XGBoost + SMOTE-ENN": tuned_probas["XGBoost"],
    "LightGBM + baseline": baseline_probas["LightGBM"],
    "LightGBM + SMOTE-ENN": tuned_probas["LightGBM"],
}

plt.figure(figsize=(8, 7))
for label in configs_order:
    fpr, tpr, _ = roc_curve(y_test, probas_all[label], pos_label=POS_LABEL)
    auc = roc_auc_score(y_test, probas_all[label])
    plt.plot(fpr, tpr, color=color_map[label], lw=2,
             label=f"{label} (AUC = {auc:.4f})")

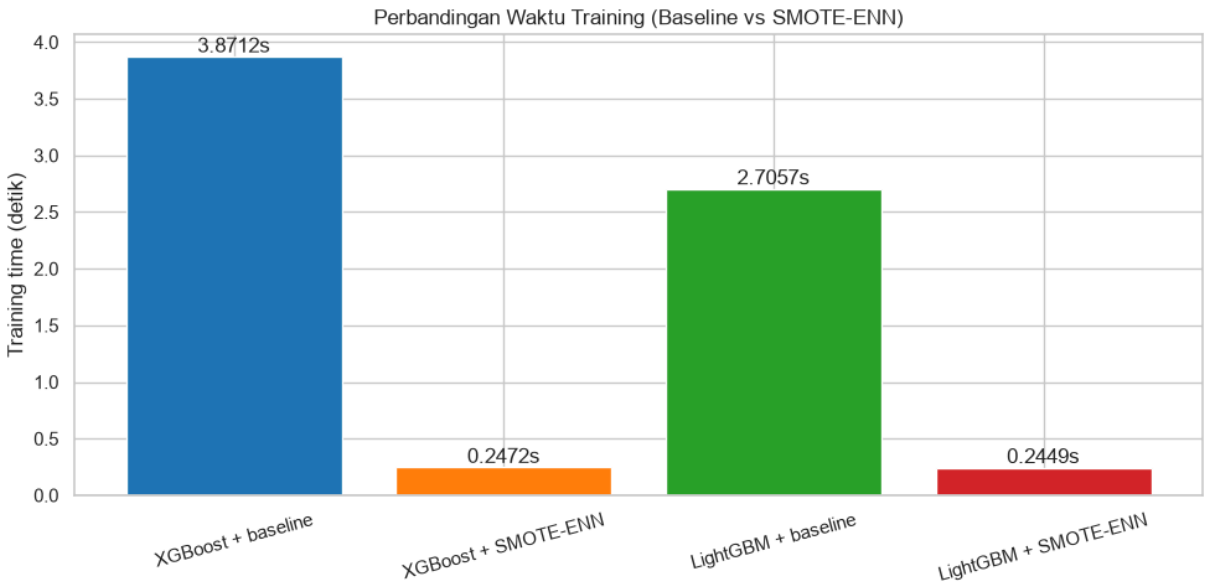
plt.plot([0, 1], [0, 1], "k--", lw=1, label="Random (AUC = 0.5)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate (Recall Stunted)")
plt.title("ROC Curve: Baseline vs SMOTE-ENN (XGBoost & LightGBM)")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



8.4 Bar Chart Waktu Training

In [45]:
```
# Perbandingan waktu training 4 kombinasi (warna konsisten dengan ROC curve)
labels_order = configs_order
times_order = [combined.set_index("label").loc[l, "train_time_s"] for l in labels_o
colors = [color_map[l] for l in labels_order]

plt.figure(figsize=(10, 5))
bars = plt.bar(labels_order, times_order, color=colors)
for bar, t in zip(bars, times_order):
    plt.annotate(f"{t:.4f}s", (bar.get_x() + bar.get_width() / 2, bar.get_height())
                 ha="center", va="bottom")
plt.ylabel("Training time (detik)")
plt.title("Perbandingan Waktu Training (Baseline vs SMOTE-ENN)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



8.5 Simpan Artefak Hasil (Opsional)

Sel berikut **opsional** — menjalankan seluruh notebook di atas sudah menghasilkan seluruh angka & visualisasi di memori. Sel ini hanya untuk menyimpan artefak ke disk bila dibutuhkan untuk laporan/Bab IV: tabel perbandingan (`CSV`) dan best hyperparameter (`JSON`) ke `results/metrics/` . Untuk mengaktifkan, ubah `SAVE_ARTIFACTS = False` → `True` lalu jalankan ulang sel ini.

In [46]:
```
# OPSIONAL: simpan artefak hasil ke results/metrics/ (default OFF)
import os, json  # hanya di-import jika penyimpanan diaktifkan

SAVE_ARTIFACTS = False
if SAVE_ARTIFACTS:
    metrics_dir = "../results/metrics"
    os.makedirs(metrics_dir, exist_ok=True)

    table.to_csv(os.path.join(metrics_dir, "master_comparison.csv"), index=False)
    best_params = {"XGBoost": xgb_gscv.best_params_, "LightGBM": lgbm_gscv.best_par
    with open(os.path.join(metrics_dir, "best_params.json"), "w") as f:
        json.dump(best_params, f, indent=2)
    print(f"Artefak tersimpan ke {metrics_dir}")
else:
    print("SAVE_ARTIFACTS = False — artefak tidak disimpan ke disk.")
```

```
SAVE_ARTIFACTS = False — artefak tidak disimpan ke disk.
```

8.6 Panduan Interpretasi (Fokus Recall & F1 Kelas Stunted)

- **Recall (Stunted)** = proporsi balita *stunting* yang berhasil dideteksi model. Nilai rendah → banyak *false negative* → anak stunting terlewat intervensi. Ini metrik terpenting penelitian ini.

- **Precision (Stunted)** = ketepatan saat model memprediksi *stunting* (menekan *false positive*).
- **F1 (Stunted)** = rata-rata harmonik precision & recall; tolok ukur keberhasilan SMOTE-ENN dalam menyeimbangkan deteksi kelas minoritas.
- **Jangan** menyimpulkan model "terbaik" dari accuracy semata — accuracy bisa tinggi meski model gagal mendeteksi banyak kasus stunting (*accuracy paradox*).

Poin analisis utama:

1. **Dampak SMOTE-ENN:** recall kelas Stunted naik tipis (XGBoost 0.9401 → 0.9454; LightGBM 0.9454 → 0.9494; FN turun 194→177 dan 177→164), dengan trade-off precision/accuracy turun tipis — sesuai karakter data yang hanya *mild imbalance* (40:60).
2. **Gap CV vs Test:** skor CV GridSearchCV (≈0.996) jauh di atas F1 test (≈0.945–0.951) — bukti bahwa CV pada data sintetis bersifat optimistik; skor CV bukan jaminan generalisasi.
3. **XGBoost vs LightGBM:** performa sangat dekat; LightGBM sedikit unggul di recall/F1 pada kedua skenario dan lebih cepat dalam training.
4. **Keterbatasan** (diakui eksplisit): baseline tidak di-tuning (perbandingan belum sepenuhnya apple-to-apple), belum ada uji signifikansi (McNemar), dan pengukuran waktu training bersifat indikatif (satu kali pengukuran per mesin).